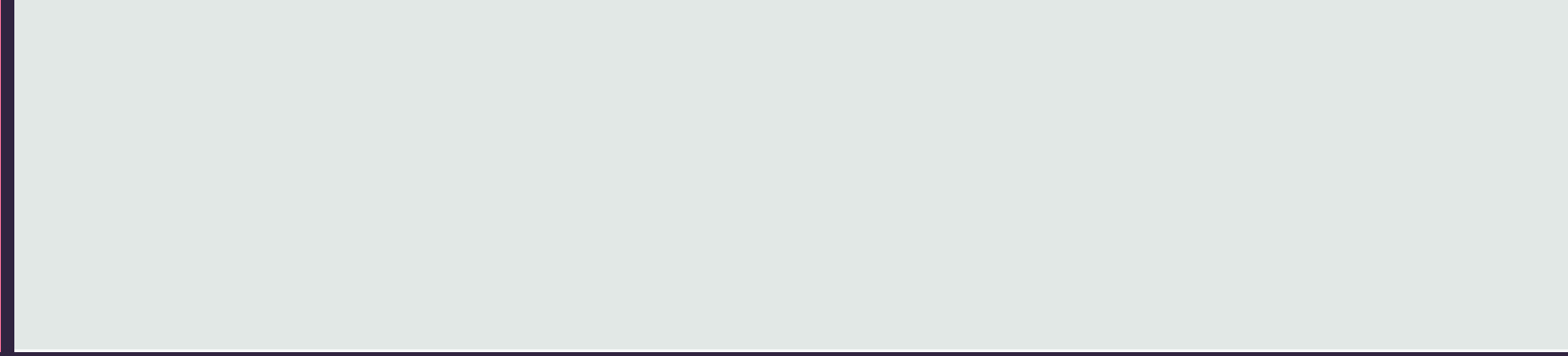




PYTHON基本語法

By Wendy Chin

On Nov.21st,2022

What Is Python

Python 是一個高層次的結合了解釋性、編譯性、互動性和物件導向的指令碼語言。

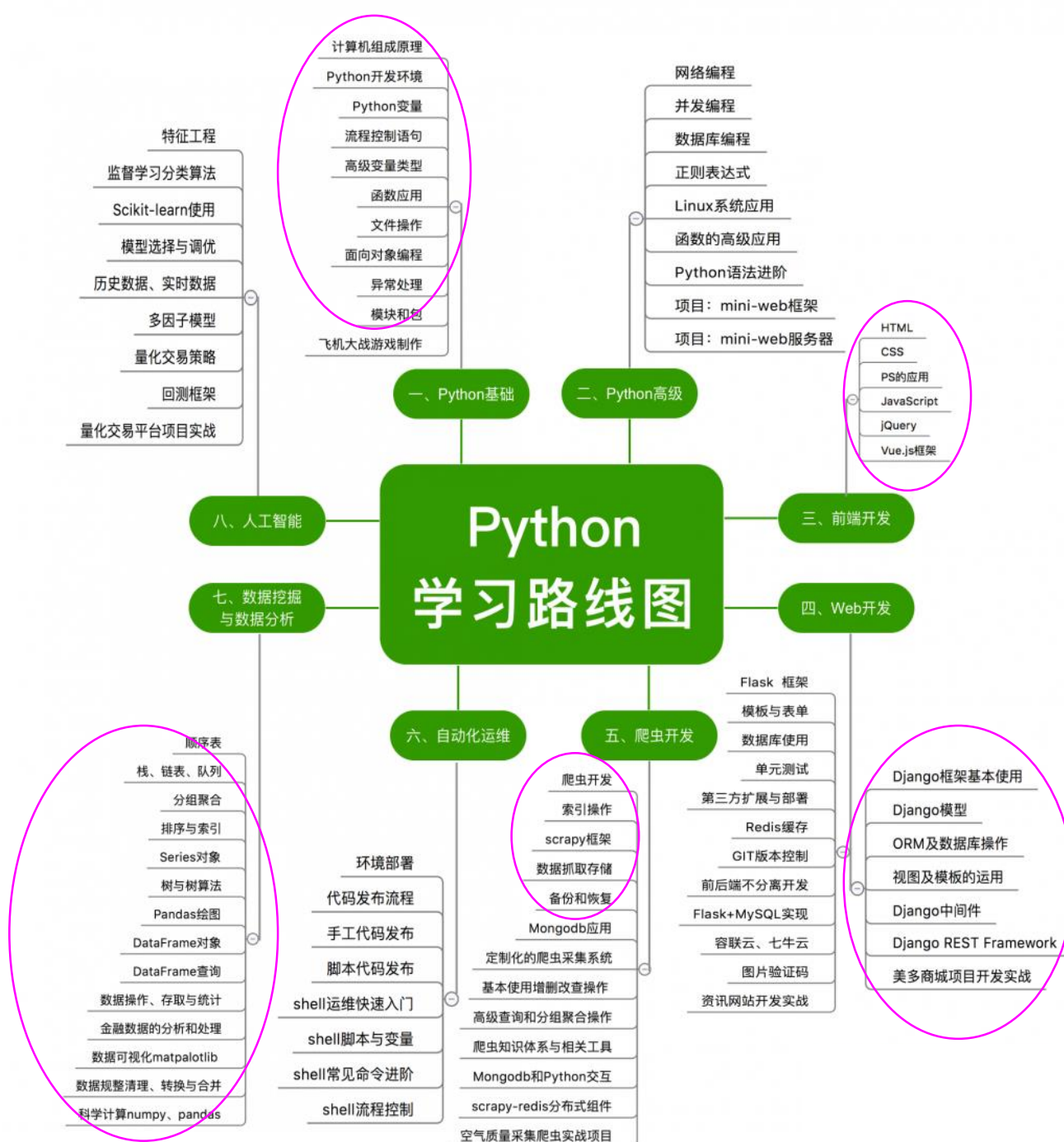
Python 的設計具有很強的可讀性，相比其他語言經常使用英文關鍵字，其他語言的一些標點符號，它具有比其他語言更有特色語法結構。

- **Python 是一種解釋型語言：**這意味著開發過程中沒有了編譯這個環節。類似於PHP和Perl語言。
- **Python 是互動式語言：**這意味著，您可以在一個 Python 提示符 `>>>` 後直接執行代碼。
- **Python 是物件導向語言：**這意味著Python支援物件導向的風格或代碼封裝在物件的程式設計技術。
- **Python 是初學者的語言：**Python 對初級程式師而言，是一種偉大的語言，它支援廣泛的應用程式開發，從簡單的文字處理到 WWW 流覽器再到遊戲

Python由荷蘭數學和計算機科學研究學會的吉多·范羅蘇姆 於1990 年代初設計，作為一門叫做ABC語言的替代品。Python提供了高效的高級數據結構，還能簡單有效地面向物件程式設計。

What to learn

- ☒ Python 基礎
- ☐ Python 高級
- ☒ 前端開發
- ☒ Web開發
- ☒ 爬蟲開發
- ☐ 服務器運維
- ☒ 數據挖掘與數據分析
- ☐ 人工智能



命名規則以及建議

- 由英文字母(a-z和A-Z)、數字(0-9)、下划線(_)來組成, 如: Name88 或 my_Name_1
- 建議以英文字母當字首
- 不建議以下划線開頭 (以下划線開頭有其它用途)
- 不能以數字開頭(容易辨識為數字), 或全都是數字
- Python裡大小寫是有區別的, 所以英文字母區別大小寫 (A和a不同)
- 不能有空格或其它符號
- 不能用保留字/關鍵字(程式保留, 有特殊用途), 也不要內置函數
- 命名時要有意義, (如: 不要取名a、aa或a1等無法識別意思的名字)

關鍵字

and	exec	not	assert	finally	or
break	for	pass	class	from	print
continue	global	raise	def	if	return
del	import	try	elif	in	while
else	is	with	except	lambda	yield

標識符-命名規則

識別字是程式設計時使用的名字，用於給變數、函數、語句塊等命名，Python 中識別字由字母、數位、底線組成，不能以數字開頭，區分大小寫。

以底線開頭的識別字有特殊含義，單底線開頭的識別字，如：_xxx，表示不能直接訪問的類屬性，需通過類提供的介面進行訪問，不能用 from xxx import * 導入；雙底線開頭的識別字，如：__xx，表示私有成員；雙底線開頭和結尾的識別字，如：__xx__，表示 Python 中內置標識，如：__init__() 表示類的構造函數。

注釋

- 單行註釋**: 用#來代表跟在後面的字符串是註釋
- 多行註釋**: 用一對三個單引號('')或三個雙引號('')包圍起來, 但一對要用相同符號, 不能混用

```
# 如下是測試內容  
a="Hello World"    #定義並賦值變量a為字符串
```

```
'''  
以下都是測試內容  
請注意變化  
'''
```

註釋是提供寫代碼說明或註解的, 提供後面的人看代碼時比較好懂在寫什麼和說明邏輯或功能, 註釋不會被Python執行, 例如, 可以單獨一行, 也可以放在句末。

“縮進” 範例：

```
14 if True:
15     print('True')
16 else:
17     print('False')
```

```
20 if True:
21     print('True')
22 else:
23     print('Answer')
24     print('false')
```

D:\Python_Test\venv\Scripts\python.exe D:/Python_Test/Test.py

File "D:\Python_Test\Test.py", line 24

print('false')

報錯：第24行縮進不一致

IndentationError: unindent does not match any outer indentation level

“行” 範例：

```
26 test='This is '+' \
27     'Python World.'
28 testlist=['This', 'Is', 'Python',
29          'World', '.']
30 print(test)
31 print(testlist)
32 print('This');print('is');print('Python World.')
```

縮進

python最具特色的就是使用縮進來表示代碼塊，不需要使用大括弧 `{}`。

縮進的空格數是可變的，但是同一個代碼塊的語句必須包含相同的縮進空格數。

行

- Python 通常是一行寫完一條語句，但如果語句很長，可以使用反斜線 `\` 來實現多行語句。
- 在 `[]`, `{}`, 或 `()` 中的多行語句，不需要使用反斜線 `\`，
- 用一行表示多個語句時，請使用分號 `;` 做區隔

數據類型

Python3 中有六个标准的数据类型：

- Number（数字），如整數、浮點數(小數), 布爾，複數
- String（字符串），用一對單引號(' ')或者雙引號("")賦值
- List（列表）,用一對中括號[] 賦值，中間用逗號隔開
- Tuple（元组），用一對小括號() 賦值，中間用逗號隔開
- Set（集合），用一對大括號 { } ，中間用逗號隔開
- Dictionary（字典）用一對大括號 { } ，中間用逗號隔開

Python3 的六个标准数据类型中：

- 不可变数据（3个）：Number（数字）、String（字符串）、Tuple（元组）；
- 可变数据（3个）：List（列表）、Dictionary（字典）、Set（集合）

```
a='This'
b='Is'
c=100

a1,b2,c2='This','Is',200.01
```

```
<class 'dict'>
<class 'list'>
<class 'tuple'>
```

```
list1=[20,23,26,30,42,89]
tuple1=(20,23,26,30,42,89)
dict1={'Dep':'PMC','Name':'A','Title':'Test1'}
print(type(dict1));print(type(list1));print(type(tuple1))
```

Python 中的變數不需要聲明。每個變數在使用前都必須賦值，變數賦值以後該變數才會被創建。

在 Python 中，變數就是變數，它沒有類型，所說的"類型"是變數所指的記憶體中物件的類型。

等號（=）用來給變數賦值。等號（=）運算符左邊是一個變數名,等號（=）運算符右邊是存儲在變數中的值。

*type()內置函數可查詢數據類型

函数	描述
<code>int(x[,base])</code>	将x转换为一个整数
<code>float(x)</code>	将x转换到一个浮点数
<code>complex(real[,imag])</code>	创建一个复数
<code>str(x)</code>	将对象 x 转换为字符串
<code>repr(x)</code>	将对象 x 转换为表达式字符串
<code>eval(str)</code>	用来计算在字符串中的有效Python表达式,并返回一个对象
<code>tuple(s)</code>	将序列 s 转换为一个元组
<code>list(s)</code>	将序列 s 转换为一个列表
<code>set(s)</code>	转换为可变集合
<code>dict(d)</code>	创建一个字典。d 必须是一个 (key, value)元组序列。
<code>frozenset(s)</code>	转换为不可变集合
<code>chr(x)</code>	将一个整数转换为一个字符
<code>ord(x)</code>	将一个字符转换为它的整数值
<code>hex(x)</code>	将一个整数转换为一个十六进制字符串

數據類型轉換

有時候，我們需要對資料內置的類型進行轉換，資料類型的轉換，一般情況下你只需要將資料類型作為函數名即可。

Python算术运算符

以下假设变量 `a=10`，变量 `b=21`：

运算符	描述	实例
+	加 - 两个对象相加	a + b 输出结果 31
-	减 - 得到负数或是一个数减去另一个数	a - b 输出结果 -11
*	乘 - 两个数相乘或是返回一个被重复若干次的字符串	a * b 输出结果 210
/	除 - x 除以 y	b / a 输出结果 2.1
%	取模 - 返回除法的余数	b % a 输出结果 1
**	幂 - 返回x的y次幂	a**b 为10的21次方
//	取整除 - 向下取接近商的整数	<pre>>>> 9//2 4 >>> -9//2 -5</pre>

```
a=10
b=21
print(a+b)
print(b%a)
print(a**b)
print(b//a)
```



```
31  
1  
10000000000000000000000000000000  
2
```

運算符-1

Python 語言支援以下類型的運算:

- 算術運算
- 比較（關係）運算
- 賦值運算
- 邏輯運算
- 位運算
- 成員運算
- 身份運算
- 運算優先順序

Python比较运算符

以下假设变量a为10，变量b为20:

运算符	描述	实例
==	等于 - 比较对象是否相等	(a == b) 返回 False。
!=	不等于 - 比较两个对象是否不相等	(a != b) 返回 True。
>	大于 - 返回x是否大于y	(a > b) 返回 False。
<	小于 - 返回x是否小于y。所有比较运算符返回1表示真，返回0表示假。这分别与特殊的变量True和False等价。注意，这些变量名的大写。	(a < b) 返回 True。
>=	大于等于 - 返回x是否大于等于y。	(a >= b) 返回 False。
<=	小于等于 - 返回x是否小于等于y。	(a <= b) 返回 True。

```
a=10
b=21
print(a+b)
print(b%a)
print(a**b)
print(b//a)
print(a==b);print(a!=b);print(a<b)
```



```
31
1
10000000000000000000000000000000
2
False
True
True
```

運算符-2

Python 語言支援以下類型的運算:

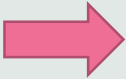
- 算術運算
- 比較（關係）運算
- 賦值運算
- 邏輯運算
- 位運算
- 成員運算
- 身份運算
- 運算優先順序

Python赋值运算符

以下假设变量a为10, 变量b为20:

运算符	描述	实例
=	简单的赋值运算符	c = a + b 将 a + b 的运算结果赋值为 c
+=	加法赋值运算符	c += a 等效于 c = c + a
-=	减法赋值运算符	c -= a 等效于 c = c - a
*=	乘法赋值运算符	c *= a 等效于 c = c * a
/=	除法赋值运算符	c /= a 等效于 c = c / a
%=	取模赋值运算符	c %= a 等效于 c = c % a
**=	幂赋值运算符	c **= a 等效于 c = c ** a
//=	取整除赋值运算符	c //= a 等效于 c = c // a
:=	海象运算符, 可在表达式内部为变量赋值。Python3.8 版本新增运算符。	在这个示例中, 赋值表达式可以避免调用 len() 两次: <pre>if (n := len(a)) > 10: print(f"List is too long ({n} elements, expected <= 10)")</pre>

```
c=a+b;print(c)  
c+=a;print(c)  
c-=a;print(c)  
c*=a;print(c)
```



```
30  
40  
30  
300
```

运算符-3

Python 語言支援以下類型的運算:

- 算術運算
- 比較（關係）運算
- 赋值運算**
- 邏輯運算
- 位運算
- 成員運算
- 身份運算
- 運算優先順序

Python逻辑运算符

Python语言支持逻辑运算符，以下假设变量 a 为 10, b为 20:

运算符	逻辑表达式	描述	实例
and	x and y	布尔"与" - 如果 x 为 False，x and y 返回 x 的值，否则返回 y 的计算值。	(a and b) 返回 20。
or	x or y	布尔"或" - 如果 x 是 True，它返回 x 的值，否则它返回 y 的计算值。	(a or b) 返回 10。
not	not x	布尔"非" - 如果 x 为 True，返回 False 。如果 x 为 False，它返回 True。	not(a and b) 返回 False

Python成员运算符

除了以上的一些运算符之外，Python还支持成员运算符，测试实例中包含了一系列的成员，包括字符串，列表或元组。

运算符	描述	实例
in	如果在指定的序列中找到值返回 True，否则返回 False。	x 在 y 序列中，如果 x 在 y 序列中返回 True。
not in	如果在指定的序列中没有找到值返回 True，否则返回 False。	x 不在 y 序列中，如果 x 不在 y 序列中返回 True。

Python身份运算符

身份运算符用于比较两个对象的存储单元

运算符	描述	实例
is	is 是判断两个标识符是不是引用自一个对象	x is y , 类似 id(x) == id(y) ，如果引用的是同一个对象则返回 True，否则返回 False
is not	is not 是判断两个标识符是不是引用自不同对象	x is not y ， 类似 id(x) != id(y) 。如果引用的不是同一个对象则返回结果 True，否则返回 False。

運算符-4

Python 語言支援以下類型的運算:

- 算術運算
- 比較（關係）運算
- 設定運算
- 邏輯運算
- 位運算
- 成員運算
- 身份運算
- 運算優先順序

运算符	描述
(expressions...),[expressions...], {key: value...}, {expressions...}	括號
x[index], x[index:index], x(arguments...), x.attribute	读取, 切片, 调用, 属性引用
await x	await 表达式
**	乘方(指数)
+x, -x, ~x	正, 负, 按位非 NOT
*, @, /, //, %	乘, 矩阵乘, 除, 整除, 取余
+, -	加和减
<<, >>	移位
&	按位与 AND
^	按位异或 XOR
	按位或 OR
in, not in, is, is not, <, <=, >, >=, !=, ==	比较运算, 包括成员检测和标识号检测
not x	逻辑非 NOT
and	逻辑与 AND
or	逻辑或 OR
if -- else	条件表达式
lambda	lambda 表达式
:=	赋值表达式

運算符-5

Python 語言支援以下類型的運算:

- 算術運算
- 比較（關係）運算
- 設定運算
- 邏輯運算
- 位運算
- 成員運算
- 身份運算
- 運算優先順序

數據結構-數值

數學函数	返回值 (描述)
abs(x)	返回数字的绝对值, 如abs(-10) 返回 10
ceil(x)	返回数字的上入整数, 如math.ceil(4.1) 返回 5
cmp(x, y)	如果 $x < y$ 返回 -1, 如果 $x == y$ 返回 0, 如果 $x > y$ 返回 1。 Python 3 已废弃, 使用 $(x > y) - (x < y)$ 替换。
exp(x)	返回e的x次幂(e^x), 如math.exp(1) 返回2.718281828459045
fabs(x)	返回数字的绝对值, 如math.fabs(-10) 返回10.0
floor(x)	返回数字的下舍整数, 如math.floor(4.9)返回 4
log(x)	如math.log(math.e)返回1.0, math.log(100,10)返回2.0
log10(x)	返回以10为基数的x的对数, 如math.log10(100)返回 2.0
max(x1, x2,...)	返回给定参数的最大值, 参数可以为序列。
min(x1, x2,...)	返回给定参数的最小值, 参数可以为序列。
modf(x)	返回x的整数部分与小数部分, 两部分的数值符号与x相同, 整数部分以浮点型表示。
pow(x, y)	$x**y$ 运算后的值。
round(x [,n])	返回浮点数 x 的四舍五入值, 如给出 n 值, 则代表舍入到小数点后的位数。(其实准确的说是保留值将保留到离上一位更近的一端。)
sqrt(x)	返回数字x的平方根。

Python 数字数据类型用于存储数值。

数据类型是不允许改变的, 这就意味着如果改变数字数据类型的值, 将重新分配内存空间

常用的內置函數如左圖

- 數學函數
- 隨機函數
- 三角函數

函数	描述
<code>choice(seq)</code>	从序列的元素中随机挑选一个元素，比如 <code>random.choice(range(10))</code> ，从0到9中随机挑选一个整数。
<code>randrange([start,] stop [,step])</code>	从指定范围内，按指定基数递增的集合中获取一个随机数，基数默认值为 1
<code>random()</code>	随机生成下一个实数，它在[0,1)范围内。
<code>seed([x])</code>	改变随机数生成器的种子seed。如果你不了解其原理，你不必特别去设定seed，Python会帮你选择seed。
<code>shuffle(lst)</code>	将序列的所有元素随机排序
<code>uniform(x,y)</code>	随机生成下一个实数，它在[x,y)范围内。

函数	描述
<code>acos(x)</code>	返回x的反余弦弧度值。
<code>asin(x)</code>	返回x的反正弦弧度值。
<code>atan(x)</code>	返回x的反正切弧度值。
<code>atan2(y, x)</code>	返回给定的 X 及 Y 坐标值的反正切值。
<code>cos(x)</code>	返回x的弧度的余弦值。
<code>hypot(x, y)</code>	返回欧几里德范数 $\sqrt{x^2 + y^2}$ 。
<code>sin(x)</code>	返回的x弧度的正弦值。
<code>tan(x)</code>	返回x弧度的正切值。
<code>degrees(x)</code>	将弧度转换为角度,如 <code>degrees(math.pi/2)</code> ， 返回90.0
<code>radians(x)</code>	将角度转换为弧度

數據結構-數值

Python 数字数据类型用于存储数值。

数据类型是不允许改变的，这就意味着如果改变数字数据类型的值，将重新分配内存空间

常用的內置函數如左圖

- 數學函數
- 隨機函數
- 三角函數

序号	内置函数	描述
1	capitalize()	将字符串的第一个字符转换为大写
2	center(width, fillchar)	返回一个指定的宽度 width 居中的字符串，fillchar 为填充的字符，默认为空格。
3	count(str, beg= 0,end=len(string))	返回 str 在 string 里面出现的次数，如果 beg 或者 end 指定则返回指定范围内 str 出现的次数
4	bytes.decode(encoding="utf-8", errors="strict")	Python3 中没有 decode 方法，但我们可以使用 bytes 对象的 decode() 方法来解码给定的 bytes 对象，这个 bytes 对象可以由 str.encode() 来编码返回。
5	encode(encoding='UTF-8',errors='strict')	以 encoding 指定的编码格式编码字符串，如果出错默认报一个ValueError 的异常，除非 errors 指定的是'ignore'或者'replace'
6	endswith(suffix, beg=0, end=len(string))	检查字符串是否以 suffix 结束，如果 beg 或者 end 指定则检查指定的范围内是否以 suffix 结束，如果是，返回 True,否则返回 False。
7	expandtabs(tabsize=8)	把字符串 string 中的 tab 符号转为空格，tab 符号默认的空格数是 8。
8	find(str, beg=0, end=len(string))	检测 str 是否包含在字符串中，如果指定范围 beg 和 end，则检查是否包含在指定范围内，如果包含返回开始的索引值，否则返回-1
9	index(str, beg=0, end=len(string))	跟find()方法一样，只不过如果str不在字符串中会报一个异常。
10	isalnum()	如果字符串至少有一个字符并且所有字符都是字母或数字则返回 True，否则返回 False
11	isalpha()	如果字符串至少有一个字符并且所有字符都是字母或中文字则返回 True, 否则返回 False
12	isdigit()	如果字符串只包含数字则返回 True 否则返回 False..
13	islower()	如果字符串中包含至少一个区分大小写的字符，并且所有这些(区分大小写的)字符都是小写，则返回 True，否则返回 False

數據結構-字符串

字符串是 Python 中最常用的資料類型。可以使用引號 (' 或 ")來創建字串。創建字串很簡單，只要為變數分配一個值即可。

常用的內置函數如左圖

Python字符串运算符

下表实例变量 a 值为字符串 "Hello", b 变量值为 "Python":

操作符	描述	实例
+	字符串连接	a + b 输出结果: HelloPython
*	重复输出字符串	a*2 输出结果: HelloHello
[]	通过索引获取字符串中字符	a[1] 输出结果 e
[:]	截取字符串中的一部分, 遵循左闭右开原则, str[0:2] 是不包含第 3 个字符的。	a[1:4] 输出结果 ell
in	成员运算符 - 如果字符串中包含给定的字符返回 True	'H' in a 输出结果 True
not in	成员运算符 - 如果字符串中不包含给定的字符返回 True	'M' not in a 输出结果 True
r/R	原始字符串 - 原始字符串: 所有的字符串都是直接按照字面的意思来使用, 没有转义特殊或不能打印的字符。原始字符串除在字符串的第一个引号前加上字母 r (可以大小写) 以外, 与普通字符串有着几乎完全相同的语法。	<pre>print(r'\n') print(R'\n')</pre>
%	格式字符串	请看下一节内容。

```
a1="This "
b1="Is "
c1="Python World "
print(a1+b1+c1)
c2=c1[0:8];print(c2)
```



```
This Is Python World
Python W
```

數據結構-字符串

字符串常見的運算符號如左圖。

列表可以被索引和截取(切片)。

切片語法：
變量[起始索引：結束索引]

索引值以 0 为开始值，-1 为从末尾的开始位置，取值範圍是左閉右開

转义字符	描述	实例
\\(在行尾时)	续行符	>>> print("line1 \ ... line2 \ line1 line2 line3")
\\	反斜杠符号	>>> print("\\") \\
\'	单引号	>>> print('\'') '
\"	双引号	>>> print("\"") "
\a	响铃	>>> print("\a") 执行后电脑有响声。
\b	退格(Backspace)	>>> print("Hello \b World!") Hello World!
\000	空	>>> print("\000")
\n	换行	>>> print("\n")
\v	纵向制表符	>>> print("Hello \v World!") Hello World!
\t	横向制表符	>>> print("Hello \t World!") Hello World!
\r	回车, 将 \r 后面的内容移到字符串开头, 并逐一替换开头部分的字符, 直至将 \r 后面的内容完全替换完成。	>>> print("Hello\rWorld!") World! >>> print('google runoob taobao\r123456') 123456 runoob taobao
\f	换页	>>> print("Hello \f World!") Hello World!
\yyy	八进制数, y 代表 0~7 的字符, 例如: \012 代表换行。	>>> print("\110\145\154\154\157\40\127\157\162\154\144\41") Hello World!
\xyy	十六进制数, 以 \x 开头, y 代表的字符, 例如: \x0a 代表换行	>>> print("\x48\x65\x6c\x6c\x6f\x20\x57\x6f\x72\x6c\x64\x21") Hello World!
\other	其它的字符以普通格式输出	

數據結構-字符串

在需要在字符中使用特殊字符時，python 用反斜線 **W** 轉義字符

數據結構-列表方法

List（列表）是 Python 中使用最频繁的数据类型,它是可變的，這是它區別於字串和元組的最重要的特點，一句話概括即：列表可以修改，而字符串和元組不能。

列表可以被索引和截取(切片)，列表被截取后返回一个包含所需元素的新列表。

切片語法：
變量[起始索引：結束索引：步長]

索引值以 0 为开始值，-1 为从末尾的开始位置，取值範圍是左閉右開

方法	描述
list.append(x)	把一个元素添加到列表的结尾，相当于 a[len(a):] = [x]。
list.extend(L)	通过添加指定列表的所有元素来扩充列表，相当于 a[len(a):] = L。
list.insert(i, x)	在指定位置插入一个元素。第一个参数是准备插入到其前面的那个元素的索引，例如 a.insert(0, x) 会插入到整个列表之前，而 a.insert(len(a), x) 相当于 a.append(x)。
list.remove(x)	删除列表中值为 x 的第一个元素。如果没有这样的元素，就会返回一个错误。
list.pop([i])	从列表的指定位置移除元素，并将其返回。如果没有指定索引，a.pop()返回最后一个元素。元素随即从列表中被移除。（方法中 i 两边的方括号表示这个参数是可选的，而不是要求你输入一对方括号，你会经常在 Python 库参考手册中遇到这样的标记。）
list.clear()	移除列表中的所有项，等于 del a[:]
list.index(x)	返回列表中第一个值为 x 的元素的索引。如果没有匹配的元素就会返回一个错误。
list.count(x)	返回 x 在列表中出现的次数。
list.sort()	对列表中的元素进行排序。
list.reverse()	倒排列表中的元素。
list.copy()	返回列表的浅复制，等于a[:]。

```
list1=[20,23,26,30,42,89]
list2=list1[1:4:2];print(list2)
```



```
[23, 30]
```


數據結構-列表函數

序号	函数
1	<u>len(list)</u> 列表元素个数
2	<u>max(list)</u> 返回列表元素最大值
3	<u>min(list)</u> 返回列表元素最小值
4	<u>list(seq)</u> 将元组转换为列表

列表常用的內置函數如左圖

字典内置函数&方法

Python字典包含了以下内置函数：

序号	函数及描述	实例
1	len(dict) 计算字典元素个数，即键的总数。	<pre>>>> tinydict = {'Name': 'Runoob', 'Age': 7, 'Class': 'First'} >>> len(tinydict) 3</pre>
2	str(dict) 输出字典，可以打印的字符串表示。	<pre>>>> tinydict = {'Name': 'Runoob', 'Age': 7, 'Class': 'First'} >>> str(tinydict) "{'Name': 'Runoob', 'Class': 'First', 'Age': 7}"</pre>
3	type(variable) 返回输入的变量类型，如果变量是字典就返回字典类型。	<pre>>>> tinydict = {'Name': 'Runoob', 'Age': 7, 'Class': 'First'} >>> type(tinydict) <class 'dict'></pre>

Key
鍵

Value
值

數據結構-字典

字典是另一種可變容器模型，且可存儲任意類型物件。字典的每個鍵值 **key=>value** 對用冒號：分割，每個對之間用逗號(,)分割，整個字典包括在大括號 {} 中。其特點如下：

- 鍵必須是唯一的，但值則不必
- 值可以取任何資料類型
- 同一個鍵不允許出現兩次。創建時如果同一個鍵被賦值兩次，後一個值會覆蓋前一個
- 鍵必須不可變，可以用數值，字符串或元組充當，但不能用列表

數據結構-字典

序号	方法	内容
1	dict.clear()	删除字典内所有元素
2	dict.copy()	返回一个字典的浅复制
3	dict.fromkeys()	创建一个新字典，以序列seq中元素做字典的键，val为字典所有键对应的初始值
4	dict.get(key, default=None)	返回指定键的值，如果键不在字典中返回default 设置的默认值
5	key in dict	如果键在字典dict里返回true，否则返回false
6	dict.items()	以列表返回一个视图对象
7	dict.keys()	返回一个视图对象
8	dict.setdefault(key, default=None)	和get()类似, 但如果键不存在于字典中，将会添加键并将值设为default
9	dict.update(dict2)	把字典dict2的键/值对更新到dict里
10	dict.values()	返回一个视图对象
11	pop(key[,default])	删除字典 key（键）所对应的值，返回被删除的值。
12	popitem()	返回并删除字典中的最后一对键和值

字典常用的方法如左圖：

常用語法：

- 創建：變量=dict()
- 訪問：變量[鍵名稱]
- 修改/新增：變量[鍵名稱：值]
- 刪除字典：del 變量
- 刪除鍵:del 變量[鍵名稱]

print()函數關鍵字end

关键字end可以用于将结果输出到同一行，或者在输出的末尾添加不同的字符。

```
a1="This "  
b1="Is "  
c1="Python World "  
print(a1+b1+c1):  
c2=c1[0:8];print(c2)  
print(a1+b1+c1,end=',')
```



```
This Is Python World  
Python W  
This Is Python World ,
```

讀取鍵盤的輸入

從標準輸入讀入一行文本，預設的標準輸入是鍵盤。

語法：input(提示信息)

```
a2=input('pls input a No:')  
b2=input('pls input a No:')  
print(a2+b2)
```



```
pls input a No:4  
pls input a No:5  
45
```

* input() 返回一个字符串，对字符串使用 split(" ") 可对其按空格切分

Python輸出

Python輸出值的方式:

- 運算式語句
- print() 函數
- 文件對象的 write() 方法，標準輸出檔可以用 sys.stdout 引用。
- 輸出的形式更加多樣，可以使用 str.format() 函數來格式化輸出值。
- 將輸出的值轉成字串，可以使用 repr() 或 str() 函數來實現。
- str(): 函数返回一个用户易读的表达式形式。
- repr(): 产生一个解释器易读的表达式形式

if基本語法

```
if condition_1:
    statement_block_1
elif condition_2:
    statement_block_2
else:
    statement_block_3
```

```
a3=input('pls input a random No:')
if int(a3)<10:
    print('The No is less than 10')
elif int(a3)<20:
    print('The No.is between 10 & 20')
else:
    print('The No is more than 20')
```

If嵌套

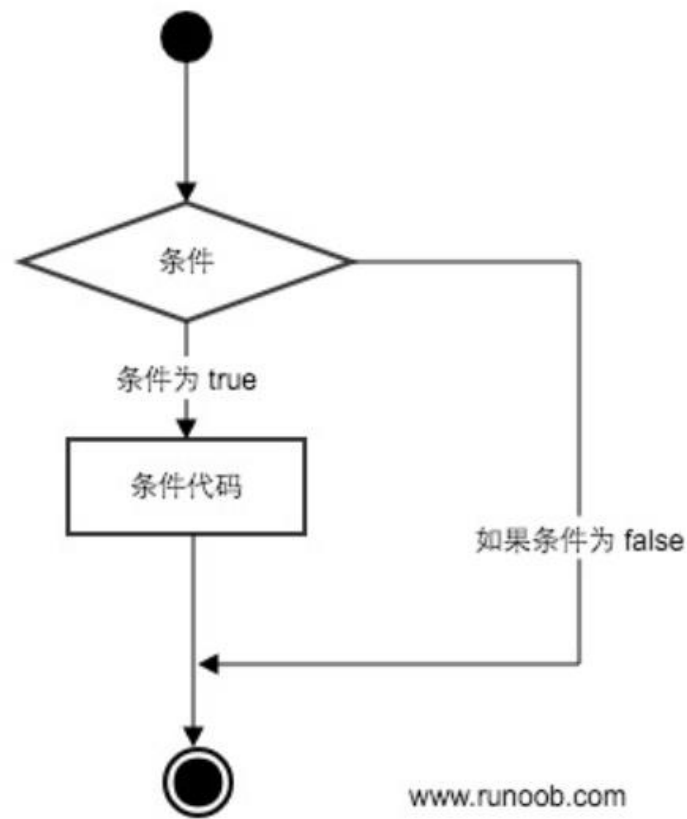
```
if 表达式1:
    语句
    if 表达式2:
        语句
    elif 表达式3:
        语句
    else:
        语句
elif 表达式4:
    语句
else:
    语句
```

Python 3.10中新增match case, 語法如下

```
match subject:
    case <pattern_1>:
        <action_1>
    case <pattern_2>:
        <action_2>
    case <pattern_3>:
        <action_3>
    case _:
        <action_wildcard>
```

條件語句-if

Python 條件陳述式是通過一條或多條語句的執行結果（True 或者 False）來決定執行的代碼塊



while基本語法

```
while <expr>:  
    <statement(s)>  
else:  
    <additional_statement(s)>
```

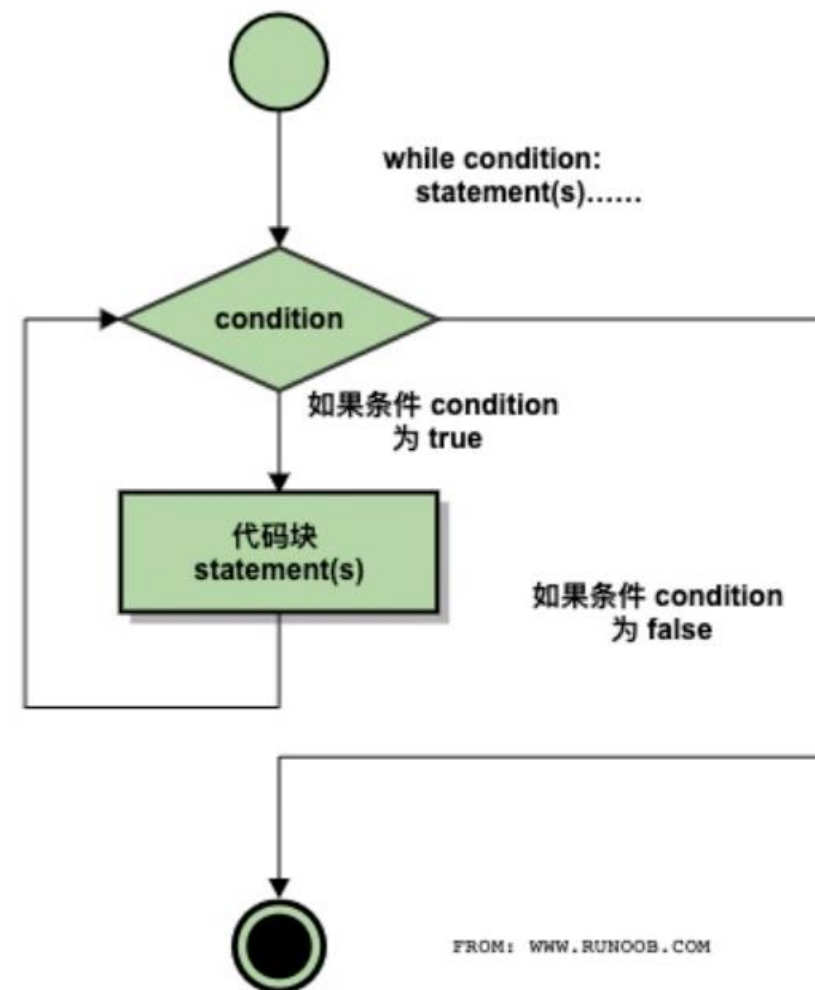
```
a4=2  
sum=11  
while sum>0:  
    sum-=a4  
    print(sum)  
else:  
    print('sum<0')
```



```
9  
7  
5  
3  
1  
-1  
sum<0
```

循環語句-while

Python 中的循環語句有 for 和 while



for基本語法

```
for <variable> in <sequence>:  
    <statements>  
else:  
    <statements>
```

```
list1=[20,23,26,30,42,89]  
sum=0  
for i in list1:  
    sum+=i  
    print(sum)  
else:  
    print('sum is 0')
```

20
43
69
99
141
230
sum is 0

for與range()搭配

語法:range(開始數字,結束數字,步長)，生成整數數列

如果只有前2個時，默認步長為1，取值範圍：左閉右開

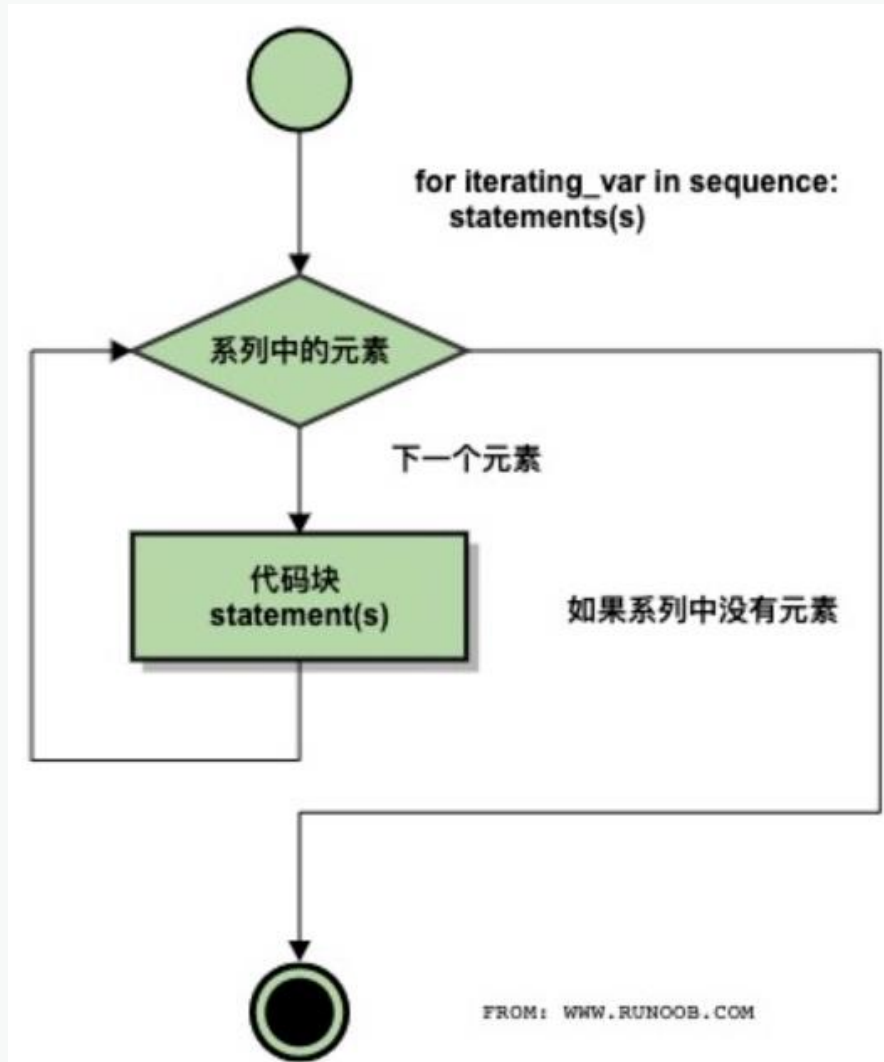
```
for i in range(1,10):  
    for x in range(1,i+1):  
        print(str(x)+" x "+str(i)+" = "+str(i*x)+"\t",end="")  
    print()
```



1 x 1 = 1									
1 x 2 = 2	2 x 2 = 4								
1 x 3 = 3	2 x 3 = 6	3 x 3 = 9							
1 x 4 = 4	2 x 4 = 8	3 x 4 = 12	4 x 4 = 16						
1 x 5 = 5	2 x 5 = 10	3 x 5 = 15	4 x 5 = 20	5 x 5 = 25					
1 x 6 = 6	2 x 6 = 12	3 x 6 = 18	4 x 6 = 24	5 x 6 = 30	6 x 6 = 36				
1 x 7 = 7	2 x 7 = 14	3 x 7 = 21	4 x 7 = 28	5 x 7 = 35	6 x 7 = 42	7 x 7 = 49			
1 x 8 = 8	2 x 8 = 16	3 x 8 = 24	4 x 8 = 32	5 x 8 = 40	6 x 8 = 48	7 x 8 = 56	8 x 8 = 64		
1 x 9 = 9	2 x 9 = 18	3 x 9 = 27	4 x 9 = 36	5 x 9 = 45	6 x 9 = 54	7 x 9 = 63	8 x 9 = 72	9 x 9 = 81	

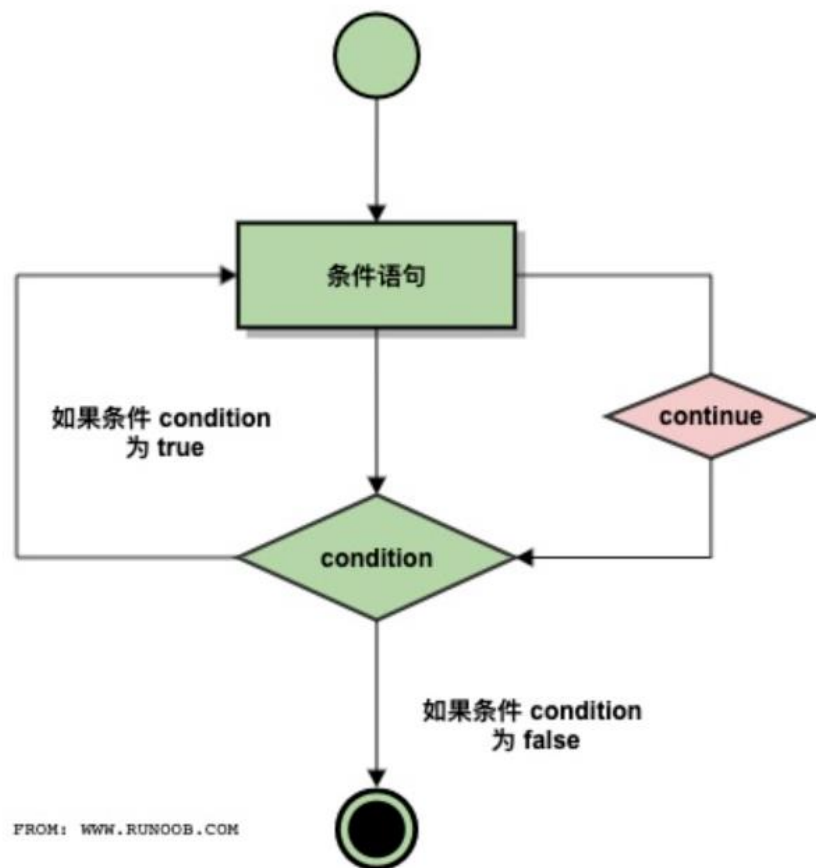
循環語句-for

Python 中的循環語句有 for 和 while



循環語句-continue

continue 语句被用来告诉 Python 跳过当前循环块中的剩余语句，然后继续进行下一轮循环



```
a4=2
sum=11
while sum>0:
    sum-=a4
    if sum==5:
        continue
    print(sum)
else:
    print('sum<0')
```

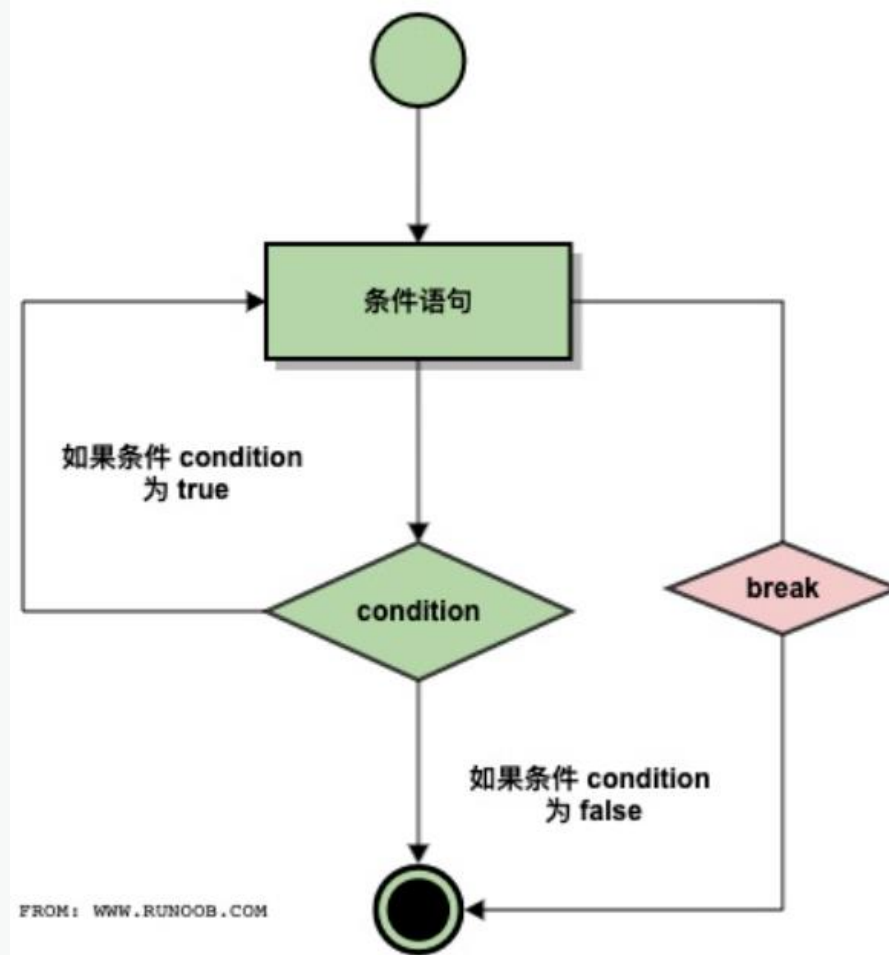
9
7
3
1
-1
sum<0

```
list1=[20,23,26,30,42,89]
sum=0
for i in list1:
    sum+=i
    if sum>100:
        break
    print(sum)
else:
    print('sum is 0')
```

20
43
69
99

循環語句-break

break 语句可以跳出 for 和 while 的循环体。如果你从 for 或 while 循环中终止，任何对应的循环 else 块将不执行



os.path() :
方法用於获取文件的属性, 具體方法如左圖

os.getcwd() :
方法用於返回當前路徑

方法	说明
os.path.abspath(path)	返回绝对路径
os.path.basename(path)	返回文件名
os.path.commonprefix(list)	返回list(多个路径)中, 所有path共有的最长的路径
os.path.dirname(path)	返回文件路径
os.path.exists(path)	路径存在则返回True,路径损坏返回False
os.path.lexists	路径存在则返回True,路径损坏也返回True
os.path.expanduser(path)	把path中包含的"~"和"~user"转换成用户目录
os.path.expandvars(path)	根据环境变量的值替换path中包含的"\$name"和"\${name}"
os.path.getatime(path)	返回最近访问时间 (浮点型秒数)
os.path.getmtime(path)	返回最近文件修改时间
os.path.getctime(path)	返回文件 path 创建时间
os.path.getsize(path)	返回文件大小, 如果文件不存在就返回错误
os.path.isabs(path)	判断是否为绝对路径
os.path.isfile(path)	判断路径是否为文件
os.path.isdir(path)	判断路径是否为目录
os.path.islink(path)	判断路径是否为链接
os.path.ismount(path)	判断路径是否为挂载点
os.path.join(path1[, path2[, ...]])	把目录和文件名合成一个路径
os.path.normcase(path)	转换path的大小写和斜杠
os.path.normpath(path)	规范path字符串形式
os.path.realpath(path)	返回path的真实路径
os.path.relpath(path[, start])	从start开始计算相对路径
os.path.samefile(path1, path2)	判断目录或文件是否相同
os.path.sameopenfile(fp1, fp2)	判断fp1和fp2是否指向同一文件
os.path.samestat(stat1, stat2)	判断stat tuple stat1和stat2是否指向同一个文件
os.path.split(path)	把路径分割成 dirname 和 basename, 返回一个元组
os.path.splitdrive(path)	一般用在 windows 下, 返回驱动器名和路径组成的元组
os.path.splitext(path)	分割路径中的文件名与拓展名
os.path.splitunc(path)	把路径分割为加载点与文件
os.path.walk(path, visit, arg)	遍历path, 进入每个目录都调用visit函数, visit函数必须有3个参数(arg, dirname, names), dirname表示当前目录的目录名, names代表当前目录下的所有文件名, args则为walk的第三个参数
os.path.supports_unicode_filenames	设置是否支持unicode路径名

文件(夾)-OS

os 模組提供了非常豐富的方法用來處理文件和文件夾。

常用的OS方法：

- os.getcwd()
- os.path()
- os.listdir(path)
- os.open()
- os.read()
- os.write()
- os.close()
- os.rename()/os.renames()
- os.remove()
- os.makedirs()

os.listdir(path) :

用於返回指定的文件夾包含的文件或子文件夾的名字的列表

os.open(file,flags[,mode]) :

用於打開一個文件，並且設置需要的打開選項，默認0777

os.read(fd,n) :

用於從文件描述符 fd 中讀取最多 n 個字節，返回包含讀取 n位字節的字符串

os.write(fd,str) :

用於写入字符串到文件描述符 fd 中. 返回实际写入的字符串长度, 目前僅僅適用於Unix系統

os.close(fd) :

用於关闭指定的文件描述符 fd

os.rename(src,dst) :

用於命名文件或文件夾，从 src 到 dst,如果dst是一个存在的路徑, 将抛出OSError

文件(夾)-OS

os 模組提供了非常豐富的方法用來處理文件和文件夾。

常用的OS方法：

- os.getcwd()
- os.path()
- os.listdir(path)
- os.open()
- os.read()
- os.write()
- os.close()
- os.rename()/os.renamees()
- os.remove()
- os.makedirs()

`os.remove(path)` :

用於刪除指定路徑的文件。如果指定的路徑是一個文件夾，將拋出`OSError`

`os.makedirs(path)` :

用於遞歸創建多層文件夾

```
import os
import shutil
import warnings

# shutil是對OS中文件操作的補充：移動、複製、打包、壓縮、解壓

warnings.filterwarnings('ignore')
path1="C:\\Users\\wendy_chin\\Desktop\\"
path2="C:\\Users\\wendy_chin\\Desktop\\"

for fileNm in os.listdir(path1):
    if fileNm.startswith('~$'):
        continue

# 刪除帶有特定擴展名的文件
if fileNm.endswith('.txt'):
    os.remove(path1+fileNm)

# 批量新增文件夾
for i in range(9):
    fileNm='Python'+str(i)
    if not os.path.exists(path1+fileNm):
        os.makedirs(path1+fileNm)
    else:
        print("The dir: "+ fileNm + " has existed!")

# 批量移動文件或文件夾
for i in range(9):
    fileNm='Python'+str(i)
    oldpath=path1+fileNm
    newpath=path2+fileNm

    if os.path.exists(path1+fileNm):
        shutil.move(oldpath,newpath)
    else:
        print("The dir: "+fileNm+"does not exist!")
```

文件(夾)-OS

`os` 模組提供了非常豐富的方法用來處理文件和文件夾。

常用的OS方法：

- `os.getcwd()`
- `os.path()`
- `os.listdir(path)`
- `os.open()`
- `os.read()`
- `os.write()`
- `os.close()`
- `os.rename()/os.renamees()`
- `os.remove()`
- `os.makedirs()`

隨機數據-random

random模組提供了非常豐富的方法用來處理隨機數據。

常用的random方法：

- random.random()
- random.randrange()
- random.choice(path)
- random.sample()
- random.uniform()
- random.randint()

```
import random
a5=random.random() # 隨機產生一個[0,1)的隨機小數
b5=random.randrange( start: 11, stop: 101, step: 2) # 隨機產生一個[11,101)步長為2的整數
c5=random.choice([23,'ab','x5']) # 隨機選擇參數中的一個元素/字符
d5=random.sample( population: [1,'a','x','c',2,3,4,5,6,'b','y','z'], k: 6)
e5=random.sample( population: 'abcxyz123456', k: 6) # 從所給的樣本中隨機生成指定數量的數據
x='abcxyz123456'
f5=random.sample(x,len(x)) #隨機打亂順序,產生新的排列順序(返回列表)
g5=random.uniform( a: 2.6, b: 12.6) #隨機產生一個指定小數區間[2.6,12.6]的小數
h5=random.randint( a: 5, b: 20) #隨機產生一個指定整數區間[5,20]的整數
print(h5)
```

以上範例中特別注意：

1.random.sample()返回一個列表

以上範例中特別注意：
open(path, "a") 若有舊同名文件時新增會疊加到同名文件中

```
# 隨機產生20行5列共100道[20,100)的小學整數減法算數訓練題,并写入txt文件
path1="C:\\Users\\wendy_chin\\Desktop\\"
for i in range(20):
    line=""
    for j in range(5):
        x=random.randint( a: 20, b: 99)
        y=random.randint( a: 20, b: 99)
        if x>=y:
            z=str(x)+" - "+str(y)+" = "+"\\t"
        else:
            z=str(y)+" - "+str(x)+" = "+"\\t"
        line=line+z
    line=line+"\\n"
    with open(path1 + "text829.txt", "a") as f:
        f.write(line)
# open()中"a"表示文件不存在則新建;文件存在時則內容加在文件末尾;
# "w"表示件不存在則新建;文件存在時則內容清空重新写入
```


Thanks

課後作業

1. Python代碼實現: 設定 $x = \text{"This is string test"}$, 請獲取 x 的字符串長度, 並分別取前6個字符串, 第4~9個字符串, 后6個字符串。
2. Python代碼實現: 請獲取鍵盤輸入的數值, 求兩個數之和。
3. 用Python代碼實現: 在桌面新建名為“Demo1” & “Demo2”的文件夾
4. 手動將本文的附件放入“Demo1”后, 用Python代碼实现: 挑選出所有文件名稱含數字的.txt文件, 移動到“Demo2”中
5. 從20個人($X_1, X_2 \dots X_{20}$)中隨機抽獎, 產生一等獎1人, 二等獎3人, 三等獎6人, 獲獎人員不可重複。